

Exploration of TransE in a Distributed Environment

Meiyan Lu¹, Lejian Liao¹, Feng Zhang², Dandan Song¹

¹School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China

²Key Laboratory of Data Engineering and Knowledge Engineering (MOE), and School of Information, Renmin University of China
lmy9559@126.com, liaolj@bit.edu.cn, fengzhang@ruc.edu.cn, sdd@bit.edu.cn

Abstract—Knowledge graph is popular in knowledge mining fields. TransE uses the structure information of triples ($\vec{e}_h + \vec{e}_r \approx \vec{e}_t$) to embed knowledge graphs into a continuous vector space, which is a very important component in knowledge representations. However, current TransE models are only implemented on single-node machines. With the explosive growth of data volumes, single-node TransE cannot meet the demand for data processing of large knowledge graphs, so a distributed TransE is urgently needed. In this poster, we propose a distributed TransE written in MPI, which can run on HPC clusters. In our experiments, our distributed TransE exhibits high-performance speedup and accuracy.

Index Terms—Distributed TransE, Knowledge Graph, MPI.

I. INTRODUCTION

Knowledge graphs are popular in recent years. Knowledge graphs use vertices and edges to represent various entities and relations among them. So far, many translation-based models have been developed for knowledge mining, and TransE is one of them which uses the structure information of triples to represent relations and entities. Due to its simplicity and effectiveness, TransE becomes an important translating embedding model for knowledge graphs.

Many parallel TransE versions have been developed. For example, Zhang and others [1] proposed a multi-threaded implementation, ParTransX, which is efficient for translating embeddings. Tay and others [2] applied random walk techniques to select triples, which can make TransE easy to be parallelized. However, none of these works is implemented for distributed environments. As the amount of knowledge graph grows, single-node TransE cannot handle knowledge graphs that have large sizes.

A distributed TransE could be a solution to the growing data issues. However, developing a distributed TransE needs to handle three challenges. First, knowledge graphs need to be distributed to different nodes in the distributed environment. Second, data transmission across different nodes needs to be considered. Third, distributed TransE should not involve too much accuracy loss.

We propose a distributed TransE, which can solve the above challenges and execute in distributed HPC platforms. The distributed TransE greatly expands the system capacity, and is able to process the problem size that previous TransE cannot handle. Our distributed TransE consists of two parts: graph partitioning and distributed computation. We evaluate our distributed TransE on ShenWei-TaihuLight HPC platform [3], and experiments show that our distributed TransE achieves high speedup with minimum accuracy loss.

II. DISTRIBUTED TRANSE

We show our distributed TransE in Figure 1, which consists of two parts: graph partitioning and distributed computation. We next show their detailed implementations.

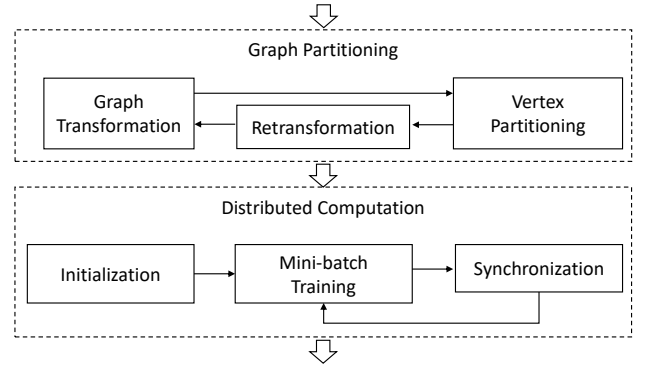


Fig. 1. Overview of our distributed TransE.

1) *Graph partitioning*: Graph partitioning is used to distribute workload to processes in distributed environments, and aims to achieve a high load balance and data locality.

- **Step 1: Graph transformation.** It transforms the original graph into a transformed one. In our work, we use SPAC [4], a popular graph transformation method, for graph transformation.
- **Step 2: Vertex partitioning.** It partitions the vertices of the transformed graph into different groups. In our work, we use KaHip [5] as our vertex partitioning strategy.
- **Step 3: Retransformation.** It retransforms the graph and provides the graph partitioning solutions according to the results of Step 2 for distributed computation of TransE.

After the above three steps, the edges are distributed to different partitions. Accordingly, a vertex may have multiple duplicates in different partitions. We randomly select a copy as the master one for storing metadata such as global states, and the rest as the ghosts. Next, we introduce the distributed computation.

2) *Distributed computation*: Distributed computation is used to perform TransE computation based on the graph partitioning in distributed environments, which can be represented as the following three steps.

- **Step 1: Initialization.** It initializes the embedding in the masters and updates the embedding in the ghosts.

- **Step 2: Mini-batch training.** It processes triples in mini-batches to model and learns the embeddings locally.
- **Step 3: Synchronization.** We use the idea of the GAS model [6] and synchronize all the embeddings in our distributed TransE.

Note that Step 2 and Step 3 execute iteratively until the model converges.

III. IMPLEMENTATION

We implement our distributed TransE in MPI, which means that our distributed TransE enjoys the applicability that it can run on various HPC environments.

IV. EVALUATION

In this section, we evaluate our distributed TransE using link prediction task [7] from performance and accuracy perspectives. As mentioned in Section II, we use SPAC [4] in graph transformation and KaHip [5] in vertex partitioning.

Experiment Setup. We perform our experiments on the ShenWei-TaihuLight HPC platform [3] with sw26010 many-core processors.

Datasets. The datasets used in our evaluation are shown in Table I. WN18 is extracted from WordNet [8] while FB15K is from Freebase [9]. The last three columns show the number of triples in train, valid, and test procedures.

TABLE I
DATASETS.

Dataset	#Entity	#Relation	Train	Valid	Test
WN18	40943	18	141442	5000	5000
FB15K	14951	1345	483142	50000	59071

Performance. To evaluate the performance, we compare the speedup ratio of our distributed TransE with a different number of processes over the original sequential TransE, as shown in Figure 2. Our distributed TransE exhibits 6.5 times speedup when the number of processes is 12 for WN18, and 9.3 times speedup when the number of processes is 20 for FB15K. The reason why the system does not reach a greater speedup may be related to the size of the dataset. We could test larger datasets in the future.

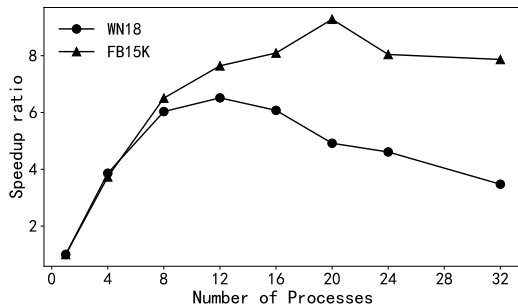


Fig. 2. Performance speedup.

Accuracy. We measure the mean rank [7] for accuracy. This metric reflects the embeddings in link prediction. For mean rank, the lower the value, the better. For WN18, the original mean rank is 228, while ours is 384; and for FB15K, the mean rank is 196, while ours is 210, which shows that our accuracy is close to the original TransE model. We also show the mean rank for different processes in Figure 3. Figure 3 shows that accuracies among different numbers of processes are close.

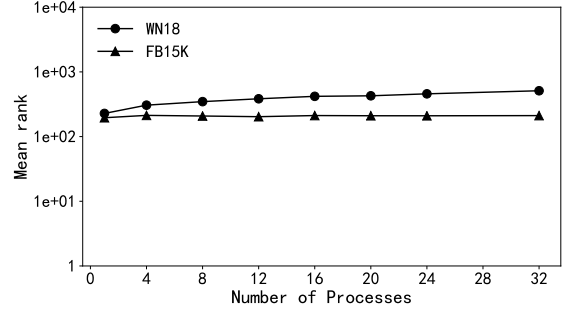


Fig. 3. Accuracy of mean rank.

V. CONCLUSION AND FUTURE WORK

In this poster, we exhibited our exploration of TransE in the distributed environment. Our distributed TransE consists of two parts: graph partitioning and distributed computation, and we have shown the detailed implementations. Experimental results show that our TransE achieves high-performance speedup while maintains high accuracy in the distributed environment. In the future, we will explore more graph transformation and vertex partitioning strategies, and use larger datasets for validation.

VI. ACKNOWLEDGMENT

This work is supported by National Natural Science Foundation of China (Grant No. 61976021) and Beijing Natural Science Foundation (Grant No. L192027). Dandan Song is the corresponding author of this paper.

REFERENCES

- [1] D. Zhang, M. Li *et al.*, "Efficient parallel translating embedding for knowledge graphs," in *Proceedings of the International Conference on Web Intelligence*, 2017.
- [2] Y. Tay, A. T. Luu, and S. C. Hui, "Non-parametric estimation of multiple embeddings for link prediction on dynamic knowledge graphs," in *AAAI*, 2017.
- [3] H. Fu, J. Liao *et al.*, "The sunway taihulight supercomputer: system and applications," *Science China Information Sciences*, vol. 59, no. 7, p. 072001, 2016.
- [4] L. Li, R. Geda *et al.*, "A simple yet effective balanced edge partition model for parallel computing," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 2017.
- [5] P. Sanders and C. Schulz, "Think locally, act globally: Highly balanced graph partitioning," in *International Symposium on Experimental Algorithms*, 2013.
- [6] J. E. Gonzalez, Y. Low *et al.*, "Powergraph: Distributed graph-parallel computation on natural graphs," in *OSDI*, 2012.
- [7] A. Bordes, N. Usunier *et al.*, "Translating embeddings for modeling multi-relational data," in *Advances in neural information processing systems*, 2013.
- [8] G. A. Miller, "Wordnet: a lexical database for english," *Communications of the ACM*, 1995.
- [9] K. Bollacker, C. Evans *et al.*, "Freebase: a collaboratively created graph database for structuring human knowledge," in *SIGMOD*, 2008.